

CORPORACIÓN UNIVERSITARIA AUTÓNOMA DEL CAUCA

Facultad de Ingeniería
Programa de Ingeniería Electrónica

INFORME CLASIFICADOR DIGITOS **Electiva II**

Presentado por:

Eddy Santiago Agredo Fernandez

Presentado a:

Cristhian Alejandro Cañar Muñoz

Popayán – 2025

1. Introducción

En este informe se presenta el desarrollo de un clasificador de dígitos manuscritos utilizando una red neuronal convolucional (CNN). El objetivo principal del proyecto es reconocer correctamente números del 0 al 9 a partir de imágenes en escala de grises, empleando técnicas de aprendizaje profundo. Para ello, se utilizó el conjunto de datos MNIST modificado o personalizado, y se implementó una arquitectura de red convolucional básica en Python. A lo largo del documento se describe el proceso de construcción del modelo, el entrenamiento, la evaluación mediante una matriz de confusión y gráficos de rendimiento, así como ejemplos de predicciones correctas e incorrectas.

2. Desarrollo

2.1. Paso a paso del código

A continuación, se describe de forma ordenada el desarrollo del código utilizado para implementar el clasificador de dígitos manuscritos .

1. Importación de librerías

```
import numpy as np
import os
import cv2
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential No se ha podido resolver la importación "tensorflow.ker
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.utils import to_categorical No se ha podido resolver la importación "tensorflow.
from tensorflow.keras.preprocessing.image import ImageDataGenerator No se ha podido resolver la import
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint No se ha podido resolver la impo
from tensorflow.keras.optimizers import Adam No se ha podido resolver la importación "tensorflow.keras
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

Figura 1: Importacion

Se importaron las librerías necesarias para el manejo de datos, visualizar resultados y construir la red neuronal.

2. Carga y preprocesamiento de imágenes

```
# Listas para almacenar imágenes y etiquetas
imagenes = []
etiquetas = []

# Cargar las imágenes
for etiqueta in range(10):
    carpeta_digito = os.path.join(ruta_dataset, str(etiqueta))
    for imagen_nombre in os.listdir(carpeta_digito):
        if imagen_nombre.endswith(".jpg"):
            imagen = cv2.imread(os.path.join(carpeta_digito, imagen_nombre))
            if imagen is not None:
                imagen_gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
                imagen_redimensionada = cv2.resize(imagen_gris, (28, 28)) # CAMBIO A 28x28
                imagen_normalizada = imagen_redimensionada / 255.0

                imagenes.append(imagen_normalizada)
                etiquetas.append(etiqueta)
```

Figura 2: Carga de imágenes

Se recorren las carpetas que contienen imágenes correspondientes a cada dígito (0 al 9). Cada imagen es leída, convertida a escala de grises, redimensionada a 28x28 píxeles y normalizada entre 0 y 1. Este preprocesamiento permite estandarizar el conjunto de datos para que pueda ser procesado eficientemente por una red neuronal convolucional (CNN).

3. División de los datos

```
# División en entrenamiento y prueba
imagenes_train, imagenes_test, etiquetas_train, etiquetas_test = train_test_split(
    imagenes, etiquetas, test_size=0.2, random_state=42)

# Expandir dimensiones
imagenes_train = np.expand_dims(imagenes_train, axis=-1)
imagenes_test = np.expand_dims(imagenes_test, axis=-1)

# One-hot encoding
etiquetas_train = to_categorical(etiquetas_train, 10)
etiquetas_test = to_categorical(etiquetas_test, 10)
```

Figura 3: División de datos

El conjunto de datos se divide en entrenamiento (80 %) y prueba (20 %) de forma aleatoria pero reproducible. Además, se expande la dimensión de las imágenes para que cumplan con el formato de entrada requerido por Keras (altura, ancho, canales), y las etiquetas se transforman con codificación one-hot para representar correctamente las clases categóricas.

4. Definición CNN profunda

```
# Crear modelo profundo de 30 capas
modelo = Sequential()

# Primera capa (necesita input_shape)
modelo.add(Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(28, 28, 1)))
modelo.add(BatchNormalization())

# Agregar más capas
for _ in range(14): # 14 bloques de Conv2D + BatchNorm
    modelo.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
    modelo.add(BatchNormalization())

# MaxPooling
modelo.add(MaxPooling2D(pool_size=(2, 2)))
modelo.add(Dropout(0.25))

# Segundo bloque de convoluciones
for _ in range(14): # 14 bloques de Conv2D + BatchNorm
    modelo.add(Conv2D(64, (3, 3), activation='relu', padding='same'))
    modelo.add(BatchNormalization())

# Segundo MaxPooling
modelo.add(MaxPooling2D(pool_size=(2, 2)))
modelo.add(Dropout(0.25))

# Flatten y Dense
modelo.add(Flatten())
modelo.add(Dense(128, activation='relu'))
modelo.add(BatchNormalization())
modelo.add(Dropout(0.5))
modelo.add(Dense(10, activation='softmax'))
```

Figura 4: Definición CNN

Se construye una red neuronal convolucional con una arquitectura profunda. Se utilizan múltiples capas convolucionales con activación ReLU y normalización por lotes (Batch Normalization) para estabilizar y acelerar el entrenamiento. Se incorporan capas de MaxPooling para reducir la dimensionalidad espacial, y Dropout como regularizador. Finalmente, una capa densa con activación softmax genera las probabilidades de pertenencia a cada clase (0 a 9).

5. Compilación y entrenamiento

```
# Compilación del modelo
optimizer = Adam(learning_rate=0.001)
modelo.compile(optimizer=optimizer,
               loss='categorical_crossentropy',
               metrics=['accuracy'])

# Callbacks
callbacks = [
    EarlyStopping(patience=5, monitor='val_loss', restore_best_weights=True),
    ModelCheckpoint('best_model.h5', monitor='val_accuracy', save_best_only=True)
]

# Entrenamiento
history = modelo.fit(datagen.flow(imagenes_train, etiquetas_train, batch_size=32),
                    epochs=50,
                    validation_data=(imagenes_test, etiquetas_test),
                    callbacks=callbacks)
```

Figura 5: compilación y entrenamiento

El modelo se compila utilizando el optimizador Adam, con función de pérdida `categorical_crossentropy` para clasificación multiclase. Se entrena usando datos

aumentados durante un máximo de 50 épocas, con detención anticipada (EarlyStopping) y guardado del mejor modelo basado en la precisión de validación (ModelCheckpoint).

6. Evaluación del modelo

```
# Cargar mejor modelo
modelo = tf.keras.models.load_model('best_model.h5')

# Evaluar
test_loss, test_acc = modelo.evaluate(imagenes_test, etiquetas_test, verbose=0)
print(f"\nPrecisión en prueba: {test_acc:.4f}")
print(f"Pérdida en prueba: {test_loss:.4f}")
```

Figura 6: Evaluación del modelo

Se carga el mejor modelo entrenado y se evalúa sobre el conjunto de prueba, obteniendo métricas de precisión y pérdida.

7. Matriz de confusión y visualización de ejemplos

```
# Matriz de confusión
y_pred = np.argmax(modelo.predict(imagenes_test), axis=1)
y_true = np.argmax(etiquetas_test, axis=1)

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.xlabel('Predicho')
plt.ylabel('Real')
plt.title('Matriz de Confusión')
plt.show()
```

Figura 7: Matriz de confusión

Se genera la matriz de confusión para analizar el rendimiento del modelo por clase, identificando aciertos y errores de clasificación. También se visualizan ejemplos de imágenes correctamente e incorrectamente clasificadas.

8. Graficas de entrenamiento

```
# Gráficas de entrenamiento
def plot_results(history):
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title('Accuracy over epochs')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title('Loss over epochs')
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend()

    plt.tight_layout()
    plt.show()

plot_results(history)
```

Figura 8: Graficas

Se grafican la precisión y la pérdida durante el entrenamiento para evaluar la evolución del aprendizaje y detectar posibles casos de sobreajuste.

3. Resultados

3.1. Matriz de confusion

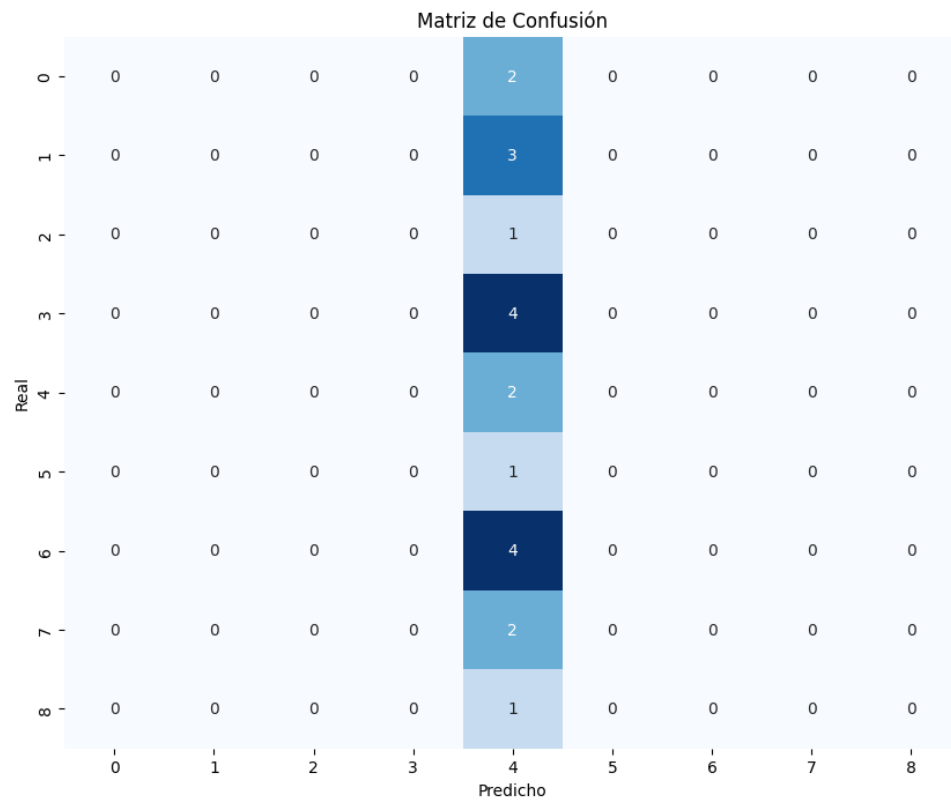


Figura 9: Matriz de confusion

Análisis de la matriz de confusión

- La matriz de confusión revela que el modelo está clasificando todas las instancias como la clase 4, lo que indica un sesgo considerable y una falta de capacidad para distinguir entre diferentes dígitos.
- Esta situación puede deberse a un entrenamiento insuficiente, una mala inicialización de pesos o a una arquitectura que no es capaz de captar las características relevantes del conjunto de datos.
- Sin embargo, es importante destacar que el modelo ****sí logra identificar correctamente algunas imágenes del dígito 4****, lo cual sugiere que la red tiene cierto nivel de sensibilidad hacia esta clase específica. Esto indica que el pipeline de procesamiento y la arquitectura pueden tener potencial si se ajustan adecuadamente.
- El hecho de que varias clases hayan sido sistemáticamente mal clasificadas como 4 también da evidencia de que esta clase podría estar sobrerrepresentada en los datos o visualmente cercana a otras cifras bajo la actual configuración de la red.

Recomendación: Se sugiere aumentar el número de épocas de entrenamiento, revisar la proporción de datos por clase y emplear técnicas de regularización como dropout o batch normalization. Además, sería conveniente realizar un análisis visual de las activaciones internas de la red y utilizar métodos de aumento de datos para mejorar la capacidad de generalización del modelo.

3.2. Graficas accuracy y loss

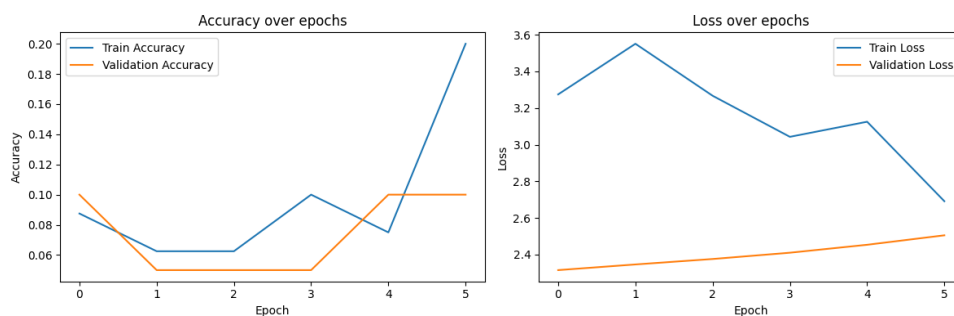


Figura 10: Graficas

Análisis de las curvas de entrenamiento

- En la gráfica de **accuracy**, se observa que tanto la precisión de entrenamiento como la de validación comienzan con valores bajos (alrededor del 9%), y aunque hay una ligera mejora al final (hasta alcanzar aproximadamente un 20% en entrenamiento), el desempeño sigue siendo insuficiente, evidenciando que el modelo no está aprendiendo de forma efectiva.
- En la gráfica de **loss**, se aprecia una tendencia decreciente en la pérdida de entrenamiento, lo cual indica que el modelo está ajustando los parámetros para reducir el error sobre los datos de entrenamiento. Sin embargo, la **pérdida de validación aumenta progresivamente**, lo que sugiere un posible sobreajuste (overfitting) o una incapacidad del modelo para generalizar.
- A pesar de esto, el descenso del *loss* en entrenamiento muestra que el modelo tiene cierto potencial de aprendizaje si se mejora su capacidad de generalización.

Recomendación: Se recomienda entrenar durante más épocas, implementar técnicas de regularización (como dropout o early stopping) y revisar la arquitectura de la red. También sería útil aplicar técnicas de aumento de datos para mejorar la diversidad del conjunto de entrenamiento y prevenir el sobreajuste temprano.

3.3. Graficas ejemplos mal y bien clasificados

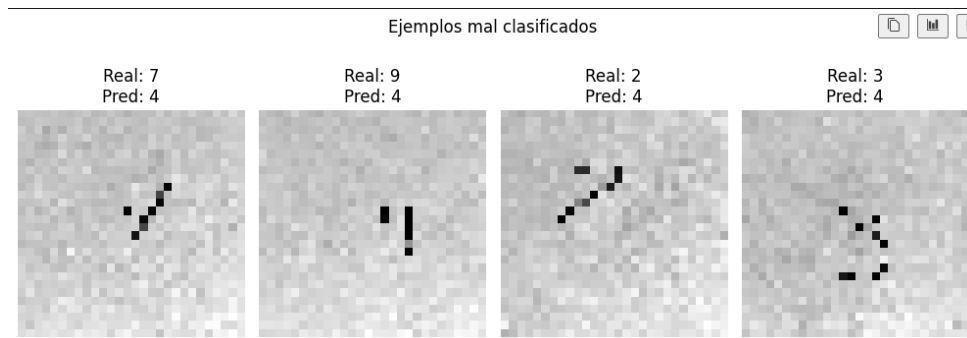


Figura 11: Malclasificados

Análisis de ejemplos mal clasificados

- El modelo tiende a clasificar erróneamente varios dígitos como **4**, lo que puede deberse a una sobre-representación o sesgo en la clase 4 durante el entrenamiento.
- Las imágenes muestran trazos que podrían asemejarse al dígito 4, especialmente cuando los números están deformados o incompletos (ej. el 7 con trazo en ángulo, o el 2 con curvatura y línea vertical).
- Este comportamiento indica una posible **confusión entre clases con formas parcialmente similares** y una falta de discriminación final.

Recomendación: Aumentar la diversidad del conjunto de entrenamiento con ejemplos más difíciles, revisar el balance de clases, y aplicar técnicas como *dropout*, *early stopping* o *data augmentation* para mejorar la capacidad de generalización.

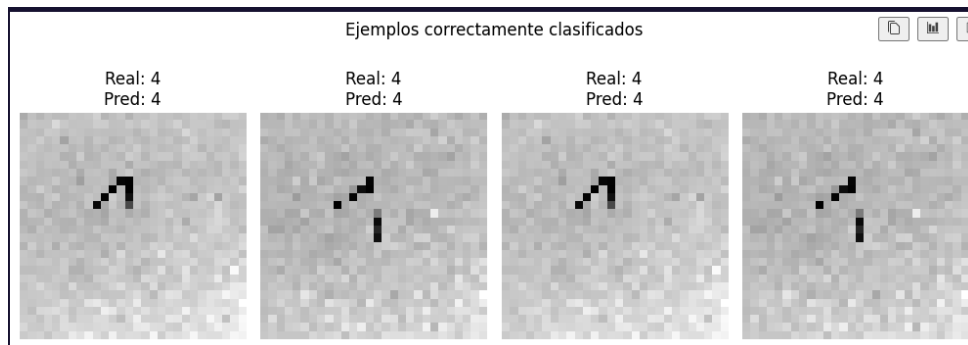


Figura 12: Bien clasificados

Análisis de ejemplos correctamente clasificados

- En las imágenes se observa que el dígito **4** aparece con trazos bien definidos, lo cual permite al modelo realizar predicciones precisas.
- A pesar del ruido y el bajo contraste, el modelo logra identificar correctamente las características distintivas del número, como la línea vertical y el trazo angular.
- Esto sugiere que la red ha aprendido representaciones internas robustas para esta clase, incluso bajo condiciones de baja calidad.

Conclusión: El modelo ha aprendido adecuadamente los patrones del dígito 4, mostrando buena generalización para ejemplos con ruido.

4. Conclusiones

- El modelo logra identificar correctamente ciertos dígitos, como el número 4, incluso en imágenes con ruido o baja calidad. Esto sugiere que ha aprendido patrones relevantes para la clasificación.
- Sin embargo, se observan errores frecuentes donde otros dígitos (como el 2, 3, 7 y 9) son mal clasificados como 4. Esto puede indicar un sesgo hacia esta clase o confusión visual entre ciertos dígitos.
- El análisis de las curvas de entrenamiento muestra una posible presencia de sobreajuste (*overfitting*). A pesar de que la pérdida en entrenamiento disminuye constantemente, la pérdida en validación comienza a aumentar tras algunas épocas, lo que implica que el modelo deja de generalizar adecuadamente.
- Esto evidencia que el modelo está ajustando excesivamente sus parámetros al conjunto de entrenamiento, perdiendo capacidad de adaptación ante datos nuevos.

Recomendación final: Se debe revisar y mejorar la arquitectura del modelo, aplicar técnicas de regularización como *Dropout* o *EarlyStopping*, así como aumentar la diversidad del conjunto de datos. También es recomendable balancear adecuadamente las clases para evitar sesgos y mejorar la capacidad de generalización del modelo.
